

From Zero to Signal

Building a Reliable Claim Prediction Pipeline
Through Data Quality, Imputation Philosophy,
and Model Selection

• • • Chronology & Troubleshooting Log

15,075

Training records

31

Feature columns

3

Modeling paradigms

| THE PROBLEM

risk_score = 0

For every single policy — complete model failure at the starting line

Missing Values

Over 8 critical features contain NaN entries — some columns missing up to 13,000+ values out of 15,075 rows.

first_claim_reported_date	13,214 missing
prior_loss_amount	1,208 missing
prior_year_mileage_000	903 missing

Categorical Inconsistency

Text columns like business_type contain case variants that the encoder treats as distinct categories.

service	Service	SERVICE
---------	---------	---------

= 3 separate buckets

Both issues corrupt the model's ability to learn signal — the scoring pipeline outputs zero because it has nothing reliable to score against. Most importantly it's a binary yes or no evaluation, $<0.5=0$.

DATA AUDIT

What the Training Data Actually Looks Like

MISSING VALUES — TRAIN.CSV (15,075 ROWS, 31 COLS)

Feature	Missing
first_claim_reported_date	13,214 (87.7%)
days_to_first_claim_report	13,214 (87.7%)
prior_loss_amount	1,208 (8.0%)
late_payment_count	1,053 (7.0%)
prior_year_mileage_000	903 (6.0%)
risk_score_external	752 (5.0%)
driver_avg_age	603 (4.0%)
payment_frequency	452 (3.0%)

TARGET DISTRIBUTION (CLAIM_COUNT)

0 claims	13,214
1 claim	1,623
2 claims	208
3+ claims	30

12.3%

True historical claim rate

Severe class imbalance — any model must account for 87.7% zero-claim policies

Three Data Philosophies

Each paradigm tests a different assumption about how missing data should be handled

PARADIGM 01

Pruned

`train_pruned.csv`

Drop any row with a missing value.
Absolute purity — every data point is authentic. Sacrifices volume for zero-noise integrity.

Trade-off: smaller dataset, zero assumptions

PARADIGM 02

Imputed

`fixed_train.csv`

Fill missing values intelligently — column means for continuous features, mode for categorical, domain logic for context-dependent fields.

Trade-off: 100% rows retained, some variance smoothed

PARADIGM 03

Sparse

Raw NaNs intact

Let the model decide.
HistGradientBoosting natively bins NaNs into whichever branch reduces loss — no imputation assumptions at all.

Trade-off: no assumptions, algorithm-constrained

PARADIGM 01

The Pruned Paradigm

Absolute purity over volume

Every row in train_pruned.csv contains only authentic, directly observed values. No assumptions, no guesses, no filled-in stand-ins. If a record was incomplete, it was removed entirely from training.

- Guarantees zero-noise, zero-assumption training data
- Reduces dataset volume — particularly rows with first_claim_reported_date
- Best use: benchmarking — cleanest possible baseline for model evaluation

ALGORITHM

Random Forest

class_weight="balanced"

MEAN PREDICTED RISK

~35%

Inflated by class balancing

OUTPUT FILE

predictions_pruned.csv

PARADIGM 02

The Artificially Completed Paradigm

Maximum volume through intelligent imputation

Rather than discarding incomplete rows, domain-specific defaults were applied to fill gaps while preserving the full 15,075-row training set.

IMPUTATION RULES

Feature	Strategy
driver_avg_age	Column mean
years_in_business	Column mean
risk_score_external	Column mean
prior_year_mileage_000	Column mean
payment_frequency	Most frequent (mode)
late_payment_count	Default to 0
prior_loss_amount	0 if no claims, else mean

100%

Rows retained

~35%

Mean predicted risk

PARADIGM 03

The Sparse Paradigm

Let the model decide what missing means

Raw NaN values are left completely intact. The HistGradientBoostingClassifier natively handles missing data by learning, for each split, whether a missing value belongs left or right — assigning it to whichever branch minimizes loss.

- No imputation assumptions — missing values are informative signals, not noise
- Naturally predicts near true baseline — no class-balancing distortion
- Restricts you to algorithms built for native NaN handling

ALGORITHM

HistGradientBoosting
Classifier

ROC-AUC SCORE

0.6687

Gini: 0.3374

MEAN RISK PREDICTED

14.90%

vs. 12.3% true baseline

The Preprocessing Pipeline

Three critical challenges solved in a unified preprocessing function

01 Target Leakage Prevention

Several columns are updated after a claim occurs — an underwriter cannot see future data when pricing a policy. So these have to be taken with a grain of salt, although they could be great predictors.

DROPPED COLUMNS

claim_paid_amount_current_period

days_to_first_claim_report

first_claim_reported_date

claim_status_current_period

total_loss_amount

02 Categorical Normalization

Case-sensitivity in text columns creates phantom categories. The pipeline standardizes all text to a single canonical form.

service

+

Service

+

SERVICE

strip whitespace → .upper()

SERVICE

OrdinalEncoder with
handle_unknown='use_encoded_value' for
unseen categories at inference time

03 Test-Time Imputation

score.csv will have its own missing values. The pipeline calculates means and modes from training data and applies those exact statistics at inference time.

Critical design choice

Train statistics never see score.csv — prevents data leakage from test distribution into the imputation baseline

KEY FINDING

The Calibration Discrepancy

Why did two sets of models predict radically different risk levels?

RANDOM FOREST

35%

Mean predicted
claim risk

Trained with
class_weight="balanced" — heavily
penalizes false negatives,
mathematically shifts all raw
probability outputs upward

vs.

HGB (SPARSE)

14.9%

Mean predicted
claim risk

No class balancing — probability
output stays close to the natural
distribution of the training data

vs.

TRUE BASELINE

12.3%

Historical claim
frequency

Ground truth — 12.3% of policies
in the training set had at least one
claim

The HGB model at 14.9% closely tracks the true 12.3% baseline — its probabilities map to real-world claim frequencies. The Random Forest's 35% output is relative risk ranking, not an absolute rate.

Unlocking the HGB Importances

Standard accuracy scoring returned flat weights — switching to ROC-AUC revealed the true signal

The Problem

HGB has no native `.feature_importances_` attribute. When permutation importance was run on binary accuracy, shuffling any feature produced near-zero impact — the heavy class imbalance meant a naive "predict all zeros" approach held accuracy constant regardless.

The Fix

Recalculated permutation importance with `scoring='roc_auc'`. This measures how shuffling a feature degrades probability confidence — not just hard binary decisions — exposing genuine feature sensitivity.

Alignment Output

All 31 original columns mapped to their Gini or permutation importances. Columns excluded from modeling are marked "Irrelevant / NO" and exported to `model_feature_importances.csv`.

TOP HGB FEATURE IMPORTANCES (ROC-AUC SCORING)

vehicle_count	30.17%
coverage_type	26.77%
risk_score_external	12.4%
prior_year_mileage_000	9.8%
driver_avg_age	7.3%
years_in_business	5.6%
late_payment_count	4.2%
prior_loss_amount	3.76%

| THE WINNER

HistGradientBoosting on Sparse Data

Highest predictive accuracy with the most realistic risk calibration

0.6687

ROC-AUC Score

0.3374

Gini Coefficient

14.9%

Mean Predicted Risk

12.3%

True Baseline Rate

Why it wins

- Best ROC-AUC across all three paradigms
- Predicts realistic frequencies — probabilities map to real-world claim rates
- No artificial inflation from class balancing — clean, interpretable output
- Native NaN handling — no imputation assumptions embedded in the model

Top drivers of prediction

vehicle_count	30.17%
coverage_type	26.77%
risk_score_external	12.40%

Output: predictions_sparse.csv

The Right Model for the Right Job

Both model families are valuable — they answer different business questions



Actuarial / Pricing

HistGradientBoosting (Sparse)

- Use for **baseline rate setting** — raw probabilities map to real claim frequencies
- 14.9% mean prediction closely mirrors the 12.3% historical baseline
- Output: predictions_sparse.csv



Underwriting / Audit

Random Forest (Pruned / Imputed)

- Use for **rank-ordering accounts** — relative risk scoring, not absolute rates
- Flag outliers and high-risk accounts for manual review — class balancing amplifies signal
- Outputs: predictions_pruned.csv / predictions_fixed.csv

| A robust, production-grade pipeline: three dataset versions, two model families, all predictions and feature importances written cleanly to `/outputs/`